BEHAVIORAL DRIFT DETECTION · MEASURED RESULTS

Deterministic drift detection: what works, what doesn't

Held-out evaluation of 11 detectors across 6 public benchmark datasets and 5,851 real user sessions. Every number is measured against a published baseline — none is argued from theory.

github.com/kydehq/
behavioral-drift-detection

Jürgen Eckel · juergen@eckel.xyz — summary of “What Does Content Access Buy?” (working draft v0.7)

TAXONOMY

Seven types of drift

Drift type	What happens
1 · Goal drift	The system leaves its assigned objective during a task.
2 · Context decay	Performance falls as the context grows; the typical failure is stopping early.
3 · Reward hacking	The system optimizes the measured proxy instead of the intended goal.
4 · Deception	Behavior under test differs from behavior in operation.
5 · Multi-agent drift	Deviation spreads between agents — e.g. inherited through handed-over transcripts.
6 · Persistent drift	Memory or self-modification makes a change durable: an incident becomes state.
7 · Version drift	The vendor swaps or updates the model; behavior changes with no operator action.

Prompt injection is a cause, not a type

A successful injection produces three of the seven, depending on where it lands:

In-session — flips the objective at once; looks like goal drift.

At hand-over — rides a transcript from one agent to the next; multi-agent drift.

Persisted — survives the session through memory; persistent drift.

The seven classes differ in cause, in what is observable, and in detection cost — they do not collapse into each other. Source: survey paper §2, Table 2.

OVERVIEW

What we tested

Which type of drift is catchable, at which logging level, by which detector?
Every cell answered by measurement on held-out data — not by argument.

7

Types of drift

The seven phenomena just shown — each with its own cause and its own detection cost.

11

Deterministic detectors

Fixed rules and statistics only. Every verdict is exactly reproducible from the logs — no LLM scoring.

4

Logging levels

From bare call metadata up to full model reasoning — each level adds data, storage cost and duties.

5,851

Real user sessions

Evaluated alongside 6 public benchmark datasets — held-out, against published baselines.

Test sets, detectors and protocol: next slides.

DATA

Seven test sets, mapped to the drift types

Test set	What it contains	Drift type exercised
Terminal Wrench	Agent trajectories labeled hacked vs. clean, three models	Reward hacking
TRACE	Independent reward-hacking corpus — used to test whether detectors transfer	Reward hacking (transfer)
AgentDojo	Tool-use runs with executed prompt injections, 29 pipelines	In-session injection — the injected form of goal drift
TRAIL	148 traces with human-annotated per-step errors	Context decay
Who&When	184 multi-agent failure runs with human failure attribution	Multi-agent drift (attribution)
AgentHallu	Attribution corpus with both failing and clean runs	Multi-agent drift (detection rates)
SWE-chat	5,851 real working sessions of 190 users, real wall-clock timestamps	Version drift · deployment fingerprinting

Two types have no public test set: deception and persistent drift.
Those records must be collected — they cannot be downloaded.

All sets are public; each is converted into a deterministic, fingerprinted ledger before evaluation. Mapping: survey paper §7.8, Table 4.

SETUP

Four logging levels, one evaluation protocol

Level	Data logged	Added cost
L0	API-call metadata: tool name, hashed parameters, status code	None — reference; ≈ 1% of raw transcript volume
L1	+ plain-text call parameters	3–4× storage; PII / redaction burden
L2	+ tool outputs (external content)	Sensitive data; input written by third parties
L3	+ model reasoning text	Largest volume; requires provider access

Evaluation protocol

All detectors are **deterministic**: fixed rules and statistics over logged data. Every verdict is exactly reproducible — no LLM-based scoring anywhere in the pipeline.

Detectors are tuned on a ~20% dev split (hash-based) and evaluated **held-out only**, with the same labels, splits and calibration as the published L0 baselines. Metric: detection rate at matched false-positive rate.

DETECTOR INVENTORY

Eleven detectors, three classes — plus a no-content baseline

D1–D3

Distribution-shift detectors

Per-run outlier score (leave-one-out likelihood) · sliding-window JSD + CUSUM change detection · block-level JSD profiles.

Requires: calibration data only. Identical code on every dataset.

D4–D9

Rule-based detectors

Hand-crafted features and heuristics: command-line rules, regex patterns over reasoning text, injection features, error signatures, prompt-diff attribution, first-error attribution.

Requires: feature engineering — and maintenance — per deployment.

D10

Consistency check

Cross-channel test: a tool-call error that the model's final answer never acknowledges.

Requires: matching tool calls to responses, per log format.

D11

Structural baselines

Position and sequence features only — no content read at all.

Requires: nothing. Twice, this baseline beat every content-based detector.

All detectors fixed before evaluation; all runs exactly reproducible. Labels D1–D11 as in the paper, §4.

CENTRAL RESULT

More logging does not buy detection. The detector class does.

Same input data — L1 command lines

Token-frequency detector (D1): **0.0%** detection at every window size.
Rule-based detector (D4): **36–73%** at ≤ 1.6% FPR.

And rules do not generalize

In-domain (tuning dataset): up to **99.7%**.
On a second dataset (TRACE): **1.5–1.9%** — chance level.

WHAT WORKS

Detectable: failures that leave machine-generated evidence in the logs

Use case	Detector	Input data	Held-out result
Version drift, batch scale	**D2**	L0 metadata	**80–100%** once JSD ≥ 0.02 (companion paper)
Reward hacking	**D4**	L1 command lines	**36–73%** at 0.5–1.6% FPR
Reward hacking, verbalized	**D5**	L3 reasoning text	**97.5–99.7%** — upper bound; assumes the agent writes out its intent
Prompt injection	**D6**	L2 tool outputs	**98.7%** at 4.5% FPR — the attack string itself is in the logged data
Failure attribution (code loop)	**D9**	L3 messages	top-1 step accuracy **30.9%** vs 12.0% random
User identification	**D3**	L0 blocks	within-user JSD **0.149** vs 0.358 between users

Common pattern: a stack trace, an exit code, an attack template, a suspicious command — evidence a machine already wrote into the logs. All results from detectors tuned and evaluated on the same dataset. Source: paper §5.

WHAT FAILS

Not detectable: generalization, ground-truth labels, and real-world variance

Where	Detector	Held-out result
Rules on a second dataset	**D4 + D5**	**1.5–1.9%** on TRACE — no generalization; chance level
Reward hacking, distribution shift	**D1, D2**	**0.0%** at L0 and L1, at every window size
Annotated reasoning errors (TRAIL)	**D7**	No signal: **6.0% vs 5.9%** on negative spans
Attribution, hand-crafted scaffolds (Who&When)	**D9 vs D11**	Content predictors **at or below** the content-free baseline; only lift is positional (18.2% vs 4.8% random)
Hallucination (AgentHallu)	**D10**	**26.3% at 21.7% FPR** — worse than chance on 3 of 7 frameworks
Real version updates (SWE-chat)	**D2**	≈ no-change control: boundary JSD **0.132** vs baseline variance 0.127
Static reference distributions	**D3**	Go stale: 0.137 → **~0.2 JSD within ~2 weeks**

These negatives cover the detector classes tested; a supervised classifier with fixed weights remains untested. Source: paper §5–§6.

WHY — THE KEY DISTINCTION

One question predicts every result in this deck

A detector can find what a machine already wrote into the logs.
It cannot judge what only the outside world knows.

The failure leaves text in the logs → detectable

A crash leaves a stack trace, a failed command a non-zero exit code, an injection its own attack string. The detector only has to match text that is already there.

- Every success in this deck is of this kind
- But each needed hand-written rules, per deployment

Judging the failure needs outside knowledge → not detectable

Is the claim true? Was the link relevant? Did the plan match the user's intent? The answer is not in the logs — they hold at best weak correlates of it.

- No tested detector beats chance at the same false-alarm budget
- More logging (L1–L3) does not change this

Not a story told after the fact: applied to the two Who&When subsets before running anything, the rule predicted which one is solvable — and was right (30.9% top-1 vs content features at zero).

REAL-WORLD CHECK — SWE-CHAT

Put to the test: 5,851 real sessions from 190 users

The method holds up, and per-deployment baselines are the right design — but subtle real-world changes drown in normal user noise.

≈ 0

The lab protocol is valid

Stitched and real chronological timelines give the same false-alarm rates. The benchmark results were no lab artifact.

2.4×

Users are distinct individuals

Each user is 2.4× closer to their own past (JSD 0.149) than to anyone else (0.358) — one baseline per deployment (here: per user) is justified.

6×

Normal noise swamps the signal

Ordinary week-to-week variation (JSD ≈ 0.13) is 6× the detection threshold. A real CLI update — version drift — vanishes in it.

2 wks

A frozen baseline expires fast

Drift from a user's first sessions reaches ~0.2 JSD within two weeks. The reference must keep updating.

JSD (Jensen–Shannon divergence): 0 = identical behavior, higher = more different. Source: paper §6.

OPERATING COSTS

What the working detectors cost to run

Content logging and rules are not a one-time buy — they charge four ongoing prices.
At realistic failure rates, most alarms would be false.

3–45×

Storage jumps a class

L0 metadata is already just ~1% of a raw transcript. Content logging multiplies that 3–45×, plus PII and redaction duties.

chance

Rules are a subscription

Hand-written rules must be redone as scaffold, task mix and language change. Carried to the next dataset unchanged, they scored at chance level.

4.5%

Other people's text sets off alarms

At L2 the detector reads third-party content. All of its false alarms were benign text that happened to look like instructions.

< 50%

In production, most alerts are wrong

Benchmarks use ~50% failures; reality is below ~5%. At that rate, precision drops under 50% — over half of all alarms are false.

4.5% is the injection detector's false-positive rate: the share of normal runs that trigger an alarm. Source: paper §5, §7.

NEXT STEPS

Three work streams, in order of expected impact

1 · SUPERVISED CLASSIFIERS

Fine-tuned BERT or gradient-boosted trees, weights fixed at inference

The untested middle ground: still fully reproducible (no test-time adaptation), and able to learn features that hand-crafted rules missed — the main blind spot.

First experiment: hallucination detection on AgentHallu and TRAIL, evaluated at matched FPR against D7/D10.

2 · ADAPTIVE BASELINES

Kalman filter / EWMA / Bayesian online change-point detection

Track the reference distribution instead of freezing it: model slow baseline evolution as state, alarm on residuals. Separates gradual variance from abrupt change.

Addresses: the 2-week staleness and the 6× variance floor — both measured. Improves the distribution-shift detectors only.

3 · BENCHMARK THE LLM JUDGE

Measure the alternative we excluded

Judge accuracy, cost per evaluation, run-to-run variance and version sensitivity — on the same datasets and splits.

Why: the build-vs-judge decision needs numbers on both sides; today only one side is measured.

Standing regression test alongside all three: apply every fixed detector unchanged to each new dataset — does it generalize? (Tested once so far; it did not.)

THE LIMIT THAT REMAINS

Labels that require external ground truth cannot be computed from log features — by rules, Kalman filters or BERTs.

Learned models improve correlates and adaptive baselines reduce false alarms; neither supplies ground truth. What deterministic detection buys, measured: failures with machine-generated evidence, per deployment, for the lifetime of the rules. Everything beyond that line is an open measurement.

Next measurement that would change this deck: a fixed-weight classifier vs D10 on AgentHallu, at matched FPR.